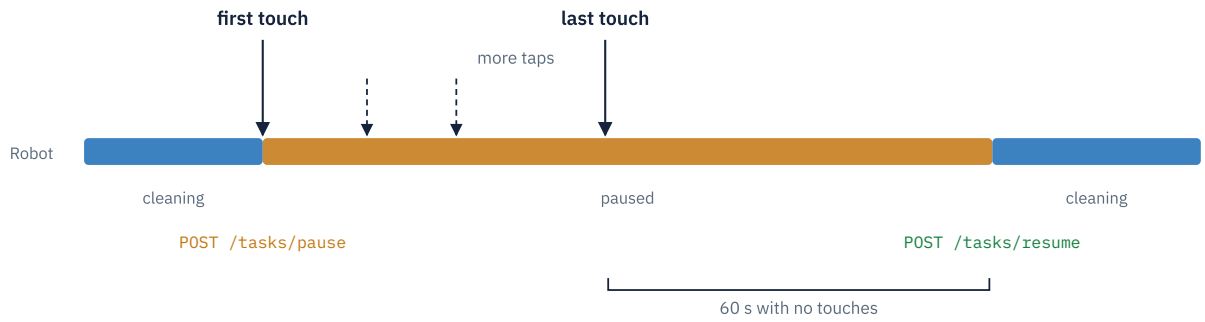
RobotApiTest Architecture

Rozylabs / robot_api_test / August 2026

Harness around `RobotPauseController`, copied out of our app unchanged (only edit: Timber became the on-screen log). Every tap is one touch event. The logic I'd like checked is in `RobotPauseController.kt`; UI and transport in `MainActivity.kt`.

What I tried to build

Touch pauses the robot; taps keep it paused; 60 seconds of quiet always resumes it, even through network failures or an app crash. The timeline below is the behavior I intended; where the app deviates from it, that deviation is what I'd like to find.



My intent: pause posted once, at the first touch of an engagement; every further tap restarts the 60-second quiet window without re-posting; resume fires when the window runs out and retries until the robot confirms it.

Connection values I used

These are the values I took from our pairing code and built this app against. If any of them don't match the robot's actual API, that alone could explain a failure. The tablet joins the robot's own hotspot (its GSM router creates the LAN); pause/resume traffic stays local and never rides GSM.

Endpoint `http://<robot-ip>:7242`	**Transport** `plain HTTP, robot's hotspot`	**Success** `any 2xx status`
Pause `POST /api/v1/tasks/pause`	**Resume** `POST /api/v1/tasks/resume`	**Request** `empty body, 3 s timeout`

If it misbehaves, my first guesses

Everything the controller does lands in the on-screen log with timestamps. For each symptom, this is where I would suspect the cause sits:

No countdown, log says pause failed

My guess: robot unreachable (wrong IP or port, tablet off the robot's hotspot, or API down). A direct `curl -X POST http://<ip>:7242/api/v1/tasks/pause` would separate my code from the network

Log says touch ignored, no paired robot

That's my guard for a URL field not starting with `http`; the controller never ran

Countdown hits 0, robot stays stopped

The log should show resume retrying; my code is sending, the robot is rejecting or not receiving `/tasks/resume`

Countdown and the log's resume time disagree

A timing bug in my controller. The countdown is a deliberately independent UI clock, so divergence means the controller's own timer is off

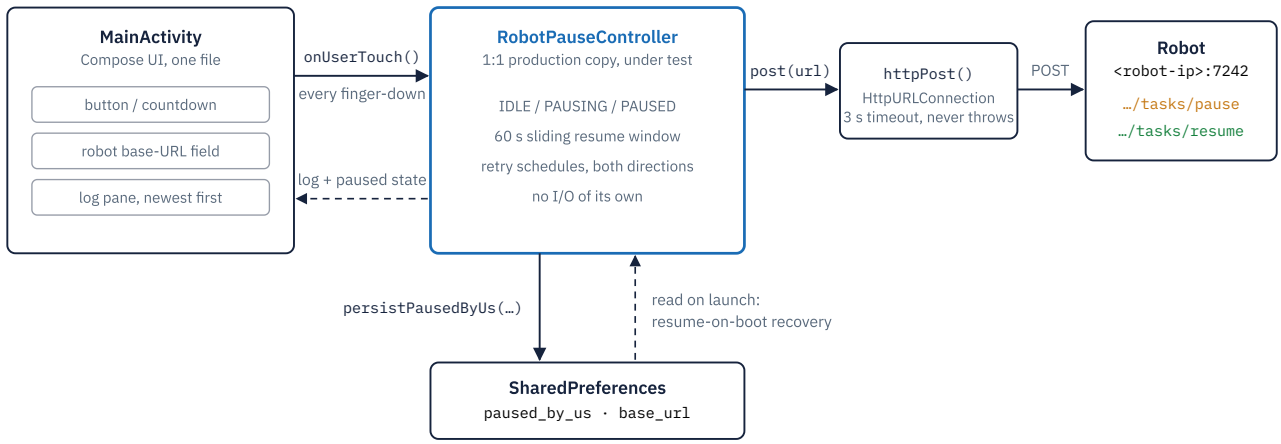
Robot moves while someone is tapping

The race I'm most worried about: a tap cancelled an in-flight resume the robot already acted on, and my pause-once rule means pause is never re-sent

Robot stuck paused after a crash

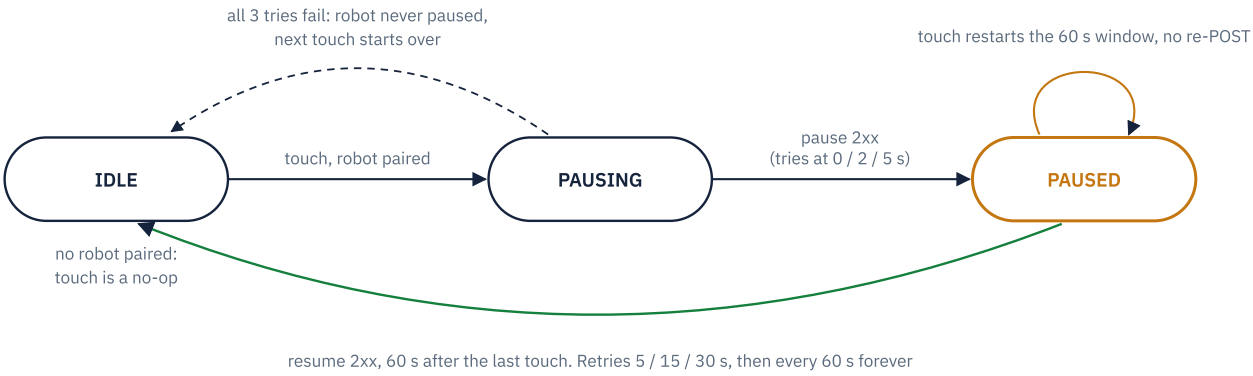
My boot recovery should post resume on the next launch, targeting the currently paired address

How the pieces talk



The controller (blue) is the production class, copied verbatim. It owns all pause and resume policy but does no I/O. Touches come in through `onUserTouch()`, and its injected lambdas are its only reach into the outside world: `target()` reads the URL field, `post()` sends the request, `persistPausedByUs()` stores the crash-recovery flag, `log()` feeds the screen. Solid arrows are calls the controller drives; dashed arrows are how the shell observes or recovers.

The controller's state machine



Pause is sent once per engagement: only the IDLE-to-PAUSING edge posts pause. While PAUSED, every touch just cancels and restarts the resume timer. The green edge is the one the design refuses to give up on. Resume retries forever, and the persisted flag replays this edge on next app launch if the process dies mid-pause.

Where I'd value scrutiny

These are the parts I'm least confident about; a second pair of eyes here helps me most.

- **Resume-cancellation race** (`restartResumeWindow()` + `resumeUntilSuccess()`): a touch can cancel an in-flight resume after the robot already acted on it, leaving the robot moving while the app thinks PAUSED. Open question: should the PAUSED touch path re-verify robot state?
- **Single-thread assumption:** `state` / `pausedTarget` / `resumeJob` have no synchronization; safe only because the injected scope is main-dispatcher. Undocumented in the class.
- **PAUSING taps:** taps during the up-to-7 s pause-retry sequence collapse into the 1-slot `SharedFlow` buffer; the one queued emission is what extends the window afterwards.
- **Faster testing:** pass a smaller `pauseWindowMs` where the controller is constructed in `MainActivity.kt` .